

Master Study Guide: Object Oriented Programming (OOP) [CT 123 / CE 123]

Unit 1: Introduction to Object-Oriented Programming - [16% of Total Marks]

A. Short Answer Questions (SAQs)

2081 End Q1 (8 Marks):

Write about the history of C++. What are the weaknesses of procedure oriented programming and the advantages of object-oriented programming?

REPEATED: Similar concept to 2079 End Q7.A and 2080 End Q7.A

2080 End Q1 (8 Marks):

Explain how the object oriented programming exhibits encapsulation? How objects-oriented programming is different than procedure oriented programming?

REPEATED: Similar concept to 2079 End Q7.A and 2081 End Q7.A

2079 End Q1 (8 Marks):

What are the advantages and disadvantages of object oriented programming? Briefly describe the features of C++.

REPEATED: Similar concept to 2080 End Q7.A and 2081 End Q1

B. Long Answer Questions (LAQs)

2081 End Q7.A (8 Marks):

Explain about class and object. Differentiate between POP and OOP.

REPEATED: Identical concept to 2079 End Q7.A and 2080 End Q1

2080 End Q7.A (8 Marks):

Explain the features of C++ and write about its history.

REPEATED: Identical concept to 2079 End Q7.A and 2081 End Q1

2079 End Q7.A (8 Marks):

Briefly explain about the history of C++. Differentiate between procedure oriented programming and object oriented programming.

REPEATED: Identical concept to 2081 End Q7.A and 2080 End Q7.A

C. Trend Analysis & Focal Areas

- **Question Types & Styles:** This unit strictly features descriptive, theory-based questions comparing POP and OOP or discussing C++ history and key features. No numerical calculations are ever asked here.
- **Direct Repetitions:**
 - POP vs. OOP differences appeared in **all three years** (2079 Q7.A, 2080 Q1, 2081 Q1 & Q7.A).
 - C++ History and Features appeared in **all three years** (2079 Q1 & Q7.A, 2080 Q7.A, 2081 Q1).
- **Key High-Priority Topics:**
 - Comprehensive structural comparison between Procedure Oriented Programming (POP) and Object Oriented Programming (OOP).
 - Core OOP concepts: Encapsulation, Abstraction, Polymorphism, and Inheritance.
 - History of C++ development and its main features.

Unit 2: C++ Language Basics & Features - [11% - 23% of Total Marks]

A. Short Answer Questions (SAQs)

2082 In-sem Q1 (5 Marks):

Explain how function polymorphism is achieved through function overloading?

REPEATED: Identical to 2081 Internal Q1

2081 Internal Q1 (7 Marks):

What do you mean by function overloading? Explain how function polymorphism is achieved through function overloading?

REPEATED: Identical to 2082 In-sem Q1

2081 End Q2 (8 Marks):

Explain the structure of C++ program. Define inline function and briefly explain its merits and demerits? Write a meaningful program to overload the volume() function. (2.5+1+1.5+3)

REPEATED: Similar concept to 2080 End Q2 and 2079 End Q2

2080 End Q2 (8 Marks):

Briefly explain the terms namespace and enumerator. In what condition the inline functions are preferred over normal functions? Write a suitable program to make inline function of a class.
(3+2+3)

REPEATED: Similar concept to 2079 End Q8.B and 2081 End Q2

2079 End Q2 (8 Marks):

Define function overloading. Write merits and demerits of inline functions. Write a meaningful program to make a user defined function that illustrates the concept of return by reference.
(1+3+4)

REPEATED: Similar concept to 2081 End Q2 and 2081 Internal Q1

B. Long Answer Questions (LAQs)

2081 Internal Q8(e) (3 Marks):

Write short note on: Dynamic memory allocation (DMA).

2081 End Q7.B (3 Marks):

Describe the namespaces in C++.

2080 End Q7.B (7 Marks):

Write syntax to allocate memory for different data types dynamically. Write a suitable program for dynamic memory allocation of any object. (2+5)

REPEATED: Similar concept to 2079 End Q8.A

2079 End Q8.A (7 Marks):

Define Dynamic Memory Allocation (DMA) and write syntax for new and delete operators. Write a program to find the smallest element of an integer array using DMA. (1+2+4)

REPEATED: Similar concept to 2080 End Q7.B

2079 End Q8.B (8 Marks):

Write about keywords, identifiers, Namespaces and manipulators. Write a program to perform the arithmetic operations like addition, subtraction, division and multiplication of three integers by making four inline functions for each arithmetic operations using concept of object and class.
(4+4)

C. Trend Analysis & Focal Areas

- **Question Types & Styles:** *This unit combines conceptual definitions (namespaces, inline functions, DMA) with small programming assignments (function overloading, dynamic memory handling, inline calculations).*
 - **Direct Repetitions:**
 - *Inline functions and their merits/demerits are heavily featured (2079 Q2, 2079 Q8.B, 2080 Q2, 2081 Q2).*
 - *Function overloading appears frequently (2079 Q2, 2081 Q2, 2081 Internal Q1, 2082 In-sem Q1).*
 - *Dynamic Memory Allocation (DMA) with **new** and **delete** is a constant source of LAQs (2079 Q8.A, 2080 Q7.B, 2081 Internal Q8).*
 - **Key High-Priority Topics:**
 - *Dynamic Memory Allocation syntax and application for arrays/objects.*
 - *Overloading functions (especially writing practical overloaded code).*
 - *Concept, syntax, and execution characteristics of inline functions.*
 - *Namespaces, manipulators, and structure of a standard C++ program.*
-

Unit 3: Classes, Objects, and Members - [13% - 22% of Total Marks]

A. Short Answer Questions (SAQs)

2082 In-sem Q2 (5 Marks):

Define constructor. Write down some characteristics of the constructor function. (1+4)

REPEATED: Similar to 2081 Internal Q2 and 2081 End Q8.A

2082 In-sem Q3 (5 Marks):

What are static variables? Write a program to illustrate the use of static data using the concept of object-oriented programming. (1+4)

REPEATED: Identical to 2081 Internal Q3

2081 Internal Q2 (7 Marks):

Write down some characteristics of the constructor function. Explain how parameters are passed implicitly to parameterized constructors using the concept of C++ programming. (4+3)

REPEATED: Similar to 2082 In-sem Q2 and 2081 End Q8.A

2081 Internal Q3 (7 Marks):

What are static variables? Define static member functions and write a program to illustrate the use of static data using the concept of object-oriented programming. (1+1+1+4)

REPEATED: Identical to 2082 In-sem Q3

2081 Internal Q6 (7 Marks):

Define the friend function and write its characteristics. Write a program to show that a friend function can act as bridge between two classes. (1+2.5+3.5)

REPEATED: Similar to 2082 In-sem Q6 and 2079 End Q7.B

2081 End Q3 (8 Marks):

Define static data and static function and write their syntaxes. Write a meaningful program to make a friend function of a class and access the class members through the friend function. (3+5)

REPEATED: Similar concept to 2080 End Q3 and 2079 End Q7.B

2080 End Q3 (8 Marks):

Write some characteristics of static data. Write a suitable program to describe the operation of static function. (4+4)

REPEATED: Similar to 2081 Internal Q3 and 2082 In-sem Q3

B. Long Answer Questions (LAQs)

2082 In-sem Q6 (10 Marks):

Define the friend function and write its characteristics. Suppose a class named Polar contains data as radius and angle (theta) and another class named Cartesian contains data as x_val and y_val, then write a meaningful C++ program to convert Cartesian type data into Polar type data. (4+6)

REPEATED: Note that the friend function definition here is 4 marks (Unit 3), while the program is 6 marks (Unit 4 Type Conversion).

2081 Internal Q8(b) (3 Marks):

Write short note on: Object and Class.

2081 Internal Q8(c) (3 Marks):

Write short note on: Destructors.

2081 End Q7.B (4 Marks):

Write some properties of the friend function.

2081 End Q8.A (8 Marks):

Write the characteristics of the constructor. Write a meaningful program to show passing objects to function. (3+5)

REPEATED: Constructor properties are highly repeated.

2080 End Q8.A (5 Marks):

Write a meaningful program to show passing of an object to function. (Note: This was part of a larger 10-mark question with virtual functions).

2079 End Q7.B (7 Marks):

What are the features of friend function? Write a meaningful program to show that the friend function acts as bridge between two classes to operate on the data of those classes of which the function is friend. (2+5)

REPEATED: Identical to 2081 Internal Q6 and 2082 In-sem Q6 (theory).

2079 End Q9.A (8 Marks):

What are the constructors and destructors? How the destructors are invoked in reverse order of constructors? Give example. (3+5)

2079 End Q9.B (7 Marks):

Define objects and classes? Why these are used in C++? Write a meaningful program to copy "engineering" as a content of an object to another object using copy constructor. (2+1+4)

C. Trend Analysis & Focal Areas

- **Question Types & Styles:** *This is a cornerstone unit. It frequently demands deep conceptual explanations (static keywords, friend properties, constructors/destructors) alongside programming tasks involving copy constructors, friend functions acting as bridges, and passing objects to functions.*
- **Direct Repetitions:**
 - *Characteristics of constructors (2081 End Q8.A, 2081 Internal Q2, 2082 In-sem Q2).*
 - *Friend function as a bridge between classes (2079 Q7.B, 2081 Internal Q6, 2082 In-sem Q6).*
 - *Static variables and static member functions (2080 Q3, 2081 Q3, 2081 Internal Q3, 2082 In-sem Q3).*
- **Key High-Priority Topics:**
 - *Constructors: Default, Parameterized, and Copy Constructors (especially writing code for copying string/object data).*
 - *Destructor execution order (how destructors execute in reverse order of construction).*
 - *Friend functions: Concept, syntax, characteristics, and practical implementation to bridge two private classes.*
 - *Static data members and static member functions.*

Unit 4: Operator Overloading and Type Conversions - [16% - 23% of Total Marks]

A. Short Answer Questions (SAQs)

2081 End Q4 (8 Marks):

List out some of the overloadable and non-overloadable operators? Write a program to overload unary plus (++) operator as a prefix operator by making overloading function as a friend function. (3+5)

REPEATED: Similar concept to 2079 End Q11.A and 2081 Internal Q7

2080 End Q4 (8 Marks):

Write a suitable program to show conversion from user defined data type to user defined data type conversion.

REPEATED: Identical to 2081 End Q9.A and 2079 End Q11.B

2080 End Q6 (8 Marks):

Write a program to overload binary plus (+) operator by making overloading function as a friend function.

REPEATED: Similar concept to 2081 End Q4

B. Long Answer Questions (LAQs)

2082 In-sem Q5 (5 Marks):

Write a program to overload unary minus (-) operator using the concept of OOP.

REPEATED: Identical to 2081 Internal Q7(b)

2082 In-sem Q6 (6 Marks):

Write a meaningful C++ program to convert Cartesian type data into Polar type data. (Sourced as part of a 10-mark compulsory question).

REPEATED: Identical to 2081 Internal Q5

2081 Internal Q5 (7 Marks):

Suppose a class named Polar contains data as radius and angle (theta) and another class named Cartesian contains data as x_val and y_val, then write a meaningful C++ program to convert Cartesian type data into Polar type data.

REPEATED: Identical to 2082 In-sem Q6

2081 Internal Q7(a) (3 Marks):

What is operator overloading? What are not overloadable operators in C++? (1+2)

2081 Internal Q7(b) (6 Marks):

Write a program to overload unary minus (-) operator using the concept of OOP.

REPEATED: Identical to 2082 In-sem Q5

2081 End Q9.A (8 Marks):

How can you convert user defined data type to user defined data type? Explain with suitable code.

REPEATED: Similar to 2080 End Q4 and 2079 End Q11.B

2080 End Q9.A (7 Marks):

Write some rules for operator overloading.

2079 End Q4 (4 Marks out of 8):

Write a program to overload unary increment operator (++) as a post fix operator. (The remaining 4 marks belong to Unit 5: Inheritance).

2079 End Q11.A (7 Marks):

What is operator overloading? Explain, how the polymorphism is achieved through operator overloading? What are not overloadable operators in C++? Give example. (1+3+3)

REPEATED: Similar to 2081 End Q4 (theory) and 2081 Internal Q7(a)

2079 End Q11.B (8 Marks):

Write a program to make two classes named apple and mango. Use concept of data conversion to convert the apple type data into mango type data.

REPEATED: Identical concept to 2081 End Q9.A and 2080 End Q4

C. Trend Analysis & Focal Areas

- **Question Types & Styles:** *This unit is heavily focused on long coding programs. You will almost certainly be asked to write a program to overload an operator (unary ++, unary -, or binary +) or perform user-defined to user-defined data conversions (such as Cartesian to Polar or Apple to Mango).*
- **Direct Repetitions:**
 - *User-defined to User-defined data conversion (2079 Q11.B, 2080 Q4, 2081 Q9.A, 2081 Internal Q5, 2082 In-sem Q6). This is the single most repeating long coding problem!*
 - *Overloading unary operators (unary ++ in 2079 Q4 & 2081 Q4; unary - in 2081 Internal Q7(b) & 2082 In-sem Q5).*
 - *List of non-overloadable operators (2079 Q11.A, 2081 Q4, 2081 Internal Q7(a)).*
- **Key High-Priority Topics:**
 - *Unary operator overloading (Prefix vs Postfix forms, regular member functions vs Friend functions).*
 - *Binary operator overloading (e.g., overloading + using friend functions).*

- *Rules of operator overloading and operators that cannot be overloaded (Scope resolution ::, Sizeof, Member selector ., Member pointer selector *. **, Conditional ?:).*
- *Type Conversion: Basic-to-User defined (via constructor), User defined-to-Basic (via casting operator), and User defined-to-User defined (via constructor in destination class or casting operator in source class).*

Unit 5: Inheritance - [15% - 23% of Total Marks]

A. Short Answer Questions (SAQs)

None directly represented in End-Semester SAQs, as inheritance is universally featured in Group B (LAQs).

B. Long Answer Questions (LAQs)

2081 Internal Q7(c) (6 Marks):

Explain Hierarchical Inheritance with its syntax and suitable example.

2081 Internal Q8(a) (3 Marks):

Write short note on: Inheritance.

2081 End Q9.B (7 Marks):

Define access specifiers and explain them. (1+6)

2081 End Q10.A (9 Marks):

Define inheritance. How do you remove ambiguity error in multiple inheritance? Give suitable example. (2+7)

REPEATED: Similar concept to 2079 End Q10.A

2081 End Q10.B (6 Marks):

Define multilevel and hybrid inheritances and write a suitable example for hybrid inheritance. (2+4)

REPEATED: Similar to 2080 End Q10.B

2080 End Q9.B (8 Marks):

Define the multipath inheritance write its syntax. Explain the hierarchical inheritance and write a program that describes its operation. (2+6)

REPEATED: Similar concept to 2079 End Q10.A

2080 End Q10.A (7 Marks):

Describe how the constructors and destructors are invoked in multiple inheritances if base and derived class have parameterized constructor and write suitable program for it.

REPEATED: Identical to 2079 End Q10.A

2080 End Q10.B (8 Marks):

Define inheritance and explain multilevel and hybrid inheritance. (1+7)

REPEATED: Identical to 2081 End Q10.B

2079 End Q4 (4 Marks out of 8):

What is inheritance? Why is it important in object oriented programming? (1+3) (The remaining 4 marks belong to Unit 4 Operator Overloading).

2079 End Q10.A (9 Marks):

What is the role of protected access specifier? Explain the hierarchical and multipath inheritance. How constructors and destructors are invoked in multiple inheritance if base and derived class have parameterized constructors. Illustrate with suitable example. (1+3+5)

REPEATED: Similar parts in 2080 End Q9.B, Q10.A and 2081 End Q9.B, Q10.A

2079 End Q10.B (6 Marks):

Define a class named Course and derive three classes as Mathematics, Science and Engineering from the class Course. Then, derive two classes from Science named: Physics and Chemistry. Define data members and member functions as appropriate. Illustrate the concept of member function overriding and accessing overridden member from the derived class in your program.

REPEATED: Spans inheritance hierarchy structures and overriding.

C. Trend Analysis & Focal Areas

- **Question Types & Styles:** This unit is always asked in the Long Answer section. Questions are highly structured, requiring both detailed theoretical explanations of types of inheritance and complex programs demonstrating inheritance hierarchy, constructors/destructors execution in multiple inheritance, and virtual base classes.
- **Direct Repetitions:**
 - Multipath inheritance and virtual base class (removing ambiguity error) is a core repeating topic (2079 Q10.A, 2080 Q9.B, 2081 Q10.A).
 - Constructor and destructor invocation in multiple inheritance (2079 Q10.A, 2080 Q10.A).

- *Definitions of multilevel and hybrid inheritance (2080 Q10.B, 2081 Q10.B).*
 - **Key High-Priority Topics:**
 - *The role of Access Specifiers: **private**, **protected**, **public** (especially protected inheritance).*
 - *Ambiguity in Multiple Inheritance and its resolution using Virtual Base Classes (Multipath inheritance).*
 - *Passing parameters to base class constructors in multiple/multilevel inheritance.*
 - *Creating structured class hierarchies (e.g., base Course deriving Math, Science, Eng).*
-

Unit 6: Polymorphism and Virtual Functions - [8% - 15% of Total Marks]

A. Short Answer Questions (SAQs)

2082 In-sem Q4 (5 Marks):

What do you mean by function overriding? Illustrate the operation of function overriding with suitable example. (1+4)

REPEATED: Identical to 2081 Internal Q4

2081 Internal Q4 (7 Marks):

What do you mean by function overriding? Why do we need function overriding? Illustrate the operation of function overriding with suitable example. (3+4)

REPEATED: Identical to 2082 In-sem Q4

2081 End Q5 (8 Marks):

Suppose the base class pointer is pointing to a derived class object and both of the classes have default constructors and destructors. Then, how can you call the destructors of both classes in a sequence? Write suitable code.

REPEATED: Similar to 2080 End Q8.B and 2079 End Q3

2079 End Q3 (8 Marks):

What is runtime polymorphism? Define abstract class and write syntax for it. How can you show that base class and child class destructors are invoked on deleting base class pointer? Illustrate with suitable example. (1+2+5)

REPEATED: Destructor sequences on deleting base pointer are highly tested.

B. Long Answer Questions (LAQs)

2081 Internal Q8(d) (3 Marks):

Write short note on: Abstract Class.

2081 End Q8.B (7 Marks):

What is a virtual function? How does it exhibit run-time polymorphism? Explain with a suitable example. (1+6)

REPEATED: Similar concept to 2080 End Q8.A

2080 End Q8.A (5 Marks):

Write some characteristics of virtual functions. (This was part of a larger 10-mark question with passing object to function).

2080 End Q8.B (5 Marks):

Write a program to illustrate the operation of virtual destructor.

REPEATED: Similar concept to 2081 End Q5 and 2079 End Q3

C. Trend Analysis & Focal Areas

- ***Question Types & Styles:*** *This unit requires both deep conceptual comprehension (runtime polymorphism vs. compile-time polymorphism) and programming capability (virtual functions, virtual destructors, pure virtual functions, and abstract classes).*
- ***Direct Repetitions:***
 - *Virtual Destructor (2079 Q3, 2080 Q8.B, 2081 Q5). This is the key coding problem here—it tests why a base class destructor must be virtual when deleting a derived class object through a base pointer.*
 - *Function Overriding (2081 Internal Q4, 2082 In-sem Q4).*
 - *Virtual functions and runtime polymorphism (2079 Q3, 2080 Q8.A, 2081 Q8.B).*
- ***Key High-Priority Topics:***
 - *Virtual Functions: Concept, characteristics, and syntax for achieving runtime polymorphism.*
 - *The crucial role of Virtual Destructors in preventing memory leaks during inheritance.*
 - *Abstract Classes and Pure Virtual Functions (syntax and application).*

- *Difference between Function Overriding and Function Overloading.*
-

Unit 7: Templates - [4% - 7% of Total Marks]

A. Short Answer Questions (SAQs)

2079 End Q5 (4 Marks out of 8):

What is function template overloading? Write a suitable program to overload function template with normal function. (1+3) (The remaining 4 marks belong to Unit 8: Exception Handling).

REPEATED: Identical to 2080 End Q11.A and 2081 End Q11.A

B. Long Answer Questions (LAQs)

2081 End Q11.A (7 Marks):

Explain how the templates follow generic programming? Explain and write a program to overload normal function with the template function. (3+4)

REPEATED: Identical to 2080 End Q11.A

2080 End Q11.A (7 Marks):

Explain how the templates follow generic programming? Explain and write a program to overload normal function with the template function. (3+4)

REPEATED: Identical to 2081 End Q11.A

C. Trend Analysis & Focal Areas

- ***Question Types & Styles:*** Consistently tested as a 7-mark question in Group B, often paired with Exception Handling (Unit 8) in Q11. It asks for the concept of generic programming and a small, complete program overloading a template function with a normal function.
- ***Direct Repetitions:***
 - *Overloading a normal function with a template function is a direct repetition in all three years (2079 Q5, 2080 Q11.A, 2081 Q11.A).*
 - *Explain how templates follow generic programming (2080 Q11.A, 2081 Q11.A).*
- ***Key High-Priority Topics:***
 - *Concept of Generic Programming and template syntax.*
 - *Function Templates vs. Class Templates.*

- *Function Template overloading with normal functions.*
-

Unit 8: Exception Handling - [4% - 8% of Total Marks]

A. Short Answer Questions (SAQs)

2079 End Q5 (4 Marks out of 8):

What are the advantages of exception handling over conventional error handling? (The remaining 4 marks belong to Unit 7: Templates).

REPEATED: Similar to 2080 End Q11.B

B. Long Answer Questions (LAQs)

2081 End Q11.B (8 Marks):

Define an exceptions and write about its types? Why do we need proper exception handling mechanism? Write an example to explain the EHM-B. (1+2+1+4)

2080 End Q11.B (8 Marks):

Define an exception. Write the advantages of exception handling mechanism over conventional error handling mechanism. Write a suitable program that explains any of the exception handling mechanism. (1+4+3)

REPEATED: Concept highly similar to 2079 End Q5 and 2081 End Q11.B

C. Trend Analysis & Focal Areas

- **Question Types & Styles:** Usually appears as an 8-mark question in Group B (paired with templates). Requires explanation of try-catch-throw mechanisms, types of exceptions, and a simple code sample demonstrating exception handling.
- **Direct Repetitions:**
 - *Advantages of exception handling over conventional error handling (2079 Q5, 2080 Q11.B, 2081 Q11.B).*
 - *Standard try-catch-throw program (2080 Q11.B, 2081 Q11.B).*
- **Key High-Priority Topics:**
 - Conventional Error Handling (returning error codes) vs. OOP Exception Handling (robust try-catch blocks).
 - The try, catch, and throw keyword syntax and execution flow.
 - Handling multiple exceptions and catch-all exceptions (**catch(...)**).

Unit 9: File Handling and Streams - [8% of Total Marks]

A. Short Answer Questions (SAQs)

2081 End Q6 (8 Marks):

What do you mean by stream class? Write a suitable program to illustrate the concept of random access of files. (1+7)

REPEATED: Similar concept to 2080 End Q5 and 2079 End Q6

2080 End Q5 (8 Marks):

What is input output stream? Explain the input output stream class hierarchy. Differentiate between the file accessing mechanisms. (1+4+3)

REPEATED: Concept similar to 2081 End Q6

2079 End Q6 (8 Marks):

Write a program to make a binary file named "student.dat" and write the data (name, roll no, email) of a student into file. Use Random access approach for writing data into the file and reading nth record from the file using concept of object and class.

REPEATED: Similar concept of random file access using objects.

B. Long Answer Questions (LAQs)

None. This unit is consistently tested as an 8-mark Group A (SAQ) question, commanding a steady 8% of the total exam weight.

C. Trend Analysis & Focal Areas

- **Question Types & Styles:** Consistently appears in Group A (Q5 or Q6) for 8 marks. It requires either theoretical class hierarchy description or standard random-access programming using files (using functions like `seekp()`, `seekg()`, `write()`, and `read()`).
- **Direct Repetitions:**
 - Random access of files (2079 Q6, 2081 Q6) is a recurring practical programming requirement.
 - C++ Stream class hierarchy and stream definitions (2080 Q5, 2081 Q6).
- **Key High-Priority Topics:**
 - C++ Standard I/O Stream Class Hierarchy (`ios`, `istream`, `ostream`, `ifstream`, `ofstream`, `fstream`).

- *Sequential vs. Random access file processing.*
- *File pointers manipulation: `seekg()`, `seekp()`, `tellg()`, `tellp()`.*
- *Writing and reading user-defined objects to/from binary files using `.write()` and `.read()`.*

SUBJECT PRIORITY PLANNER

The table below ranks the syllabus units from highest return-on-investment (ROI) to lowest. Use this to focus your limited study time for maximum marks.

Rank	Unit No.	Unit Name	Exam Priority Class	Historical Mark Percentage	Core Focus Areas to Pass/Score
1	Unit 4	Operator Overloading & Type Conversion	Highest	16% - 23%	Unary/binary overloading programs, User-Defined to User-Defined conversions (Cartesian ↔ Polar, Apple ↔ Mango).
2	Unit 5	Inheritance	Highest	15% - 23%	Multipath inheritance ambiguity resolution via Virtual Base Classes, Parameterized constructor execution order.
3	Unit 2	C++ Language Basics & Features	High	11% - 23%	Dynamic Memory Allocation (DMA) syntax and array programs, Inline function merits/demerits, Function overloading.
4	Unit 3	Classes, Objects, and Members	High	13% - 22%	Constructor characteristics, Friend function as a bridge between classes, Static variables/member functions.
5	Unit 1	Introduction to OOP	Medium	16%	Differences between POP and OOP (weaknesses/strengths), History of C++, Basic core features (encapsulation, etc.).
6	Unit 6	Polymorphism & Virtual Functions	Medium	8% - 15%	Virtual Destructors sequence of execution, runtime polymorphism, function overriding vs overloading.
7	Unit 9	File Handling and Streams	Low-Medium	8%	Binary files reading/writing, Random access (<code>seekp/seekg</code>), Stream class hierarchy.
8	Unit 8	Exception Handling	Low	4% - 8%	Try-catch-throw programs, Exception handling advantages over conventional, Types of exceptions.
9	Unit 7	Templates	Low	4% - 7%	Overloading a normal function with a template function, generic programming concepts.